

Multi-Source Candidate Data Transformer

Technical Design — one trustworthy profile per candidate, from many messy sources

Goal. Ingest messy candidate data from many sources → emit **one** canonical profile per candidate: normalized, deduplicated, with per-field **provenance** and a **confidence** score. A runtime **config** reshapes the output (a projection layer) with no engine changes. The system is **deterministic**, **robust** (a garbage source never crashes the run; unknowns become `null`, never invented), **explainable**, and scales to thousands of candidates.

PIPELINE

detect → extract → normalize → match → merge → confidence → project → validate

1. **detect** — route each input to its adapter (CLI flag, or `--input` auto-sniffed by extension/content). `github` / `linkedin` also take a profile URL or bare username and live-fetch it; any source — file or live — that's unreadable, malformed, or unreachable degrades to an empty record (skipped, logged) rather than crashing.
2. **extract** — each adapter parses its raw input into `SourceRecord`s: partial profiles in *canonical* field names plus a per-claim provenance entry `{field, source, method, raw}`. The ATS adapter remaps its own field names to ours via an explicit mapping table.
3. **normalize** — pure, deterministic functions per field map raw → canonical format. Un-normalizable input returns `None`; the raw value is preserved in provenance, never guessed.
4. **match** — cluster records sharing a strong identifier (email, phone, or profile URL) into one person.
5. **merge** — per field, pick a winner among competing claims; list fields are deduplicated unions.
6. **confidence** — score each field claim and the overall profile.
7. **project** — build the requested output from the canonical record + config (path DSL, rename, per-field normalize, toggles, `on_missing`).
8. **validate** — check the projected output against the config's declared schema before returning it.

The internal **canonical record** is built once and never mutated; **projection** is a separate read-only layer. Clean separation is the core architectural rule.

CANONICAL SCHEMA & NORMALIZED FORMATS

`candidate_id` (sha1 of the cluster's distinct match keys — deduped to a set first, so a repeated key from an extra source can't change the id), `full_name`, `emails[]`, `phones[]` (**E.164**, configurable region), `location` {city, region, country} (country **ISO-3166 alpha-2**), `links` {linkedin, github, portfolio, other[]}, `headline` | `null`, `years_experience` | `null`, `skills` [{name, confidence, sources[]}] (**canonical names** via alias map; unknown skill kept as-is, lower confidence), `experience` [{company, title, start, end, summary}] (**dates YYYY-MM**; "present" → `null` end), `education` [{institution, degree, field, end_year}], `provenance` [{field, source, method}], `overall_confidence`.

MATCH POLICY

Keys = normalized **email**, **E.164 phone**, and a **canonicalized profile URL** (LinkedIn/GitHub handle) — each ~1:1 with a person. Records sharing any key are unioned via connected components: transitive, order-independent. **No fuzzy name/company/skill matching** — a wrong merge produces a "wrong-but-confident" profile, the worst outcome named in the brief — so we stay exact; the URL key lets an email-less live fetch (e.g. GitHub with no public email) still link via a resume/notes mention. A keyless record stands alone. *Known risk*: a shared generic phone can over-merge; we accept exact-match only and document it.

MERGE & CONFIDENCE

A fixed **trust table** `trust(source, field_category) ∈ [0,1]` encodes field-source affinity (contact ← CSV/ATS; experience/headline ← LinkedIn/resume; skills ← GitHub/resume). A claim's confidence is $c_i = \text{trust} \times \text{method_factor}$ (`direct_field=1.0`, `regex_extract=0.8`, `inferred=0.6`). **Winner** = $\max c_i$; ties broken deterministically (trust → fixed source order → value sort) — **except location**, ranked by completeness first ("San Francisco, CA, US" beats "SF Bay Area" even from a higher-trust source), since sub-fields are never stitched across sources. List fields are deduplicated unions; each skill scored independently. **Confidence of the winning value**: agreeing claims combine by noisy-OR $c = 1 - \prod(1 - c_i)$; a conflict penalty then applies $c \times= (1 - 0.5 \cdot \text{disagree_fraction})$. **overall_confidence** = equal-weighted mean over {name, email, phone, skills, experience}; a missing field contributes 0. Values clamp to `[0,1]`, rounded to 2 dp. Every competing claim is kept in provenance.

RUNTIME CONFIG (PROJECTION + VALIDATION)

`Config = { fields: [{path, from?, type, required?, normalize?, on_missing?}], include_provenance, include_confidence, on_missing }`. `from` is a small path DSL: dotted paths, `[i]` index, and `[]` to map over an array extracting a field (`emails[0]`, `skills[].name`). A per-field `normalize` re-applies a normalizer at projection time. `on_missing` is `null` | `omit` | `error`, set globally with a per-field override. Validation checks each output field's declared `type` (`string`, `string[]`, `number`, object shapes) and `required` flag. The engine never changes — only the config reshapes the output.

EDGE CASES

1. **Conflicting values** (two phones) → trust-picked winner, both kept in provenance, confidence penalized.
2. **Garbage/unreachable source** (broken JSON, empty CSV, missing file, blocked live fetch) → skipped with a warning, run completes; a record left with no name/email/phone is dropped as unidentifiable noise.
3. **Duplicate person** across CSV + LinkedIn + resume → merged into one record via email/phone/profile URL.
4. **Field absent from every source** → `null`, confidence 0, demonstrating the `on_missing` policy.
5. **Un-normalizable value** (phone "call me", date "summer 2020", unknown skill) → field `null` or kept-raw, low confidence, never invented.

DETERMINISM

Fixed trust tables, stable sort orders (skills, emails), `candidate_id` from a content hash, and no wall-clock or RNG in the output → byte-stable output for identical inputs. A gold-profile test asserts this.

SCOPE & KNOWN LIMITS (STATED HONESTLY)

Descoped under time pressure: heavy PDF/DOCX layout parsing (plain prose only); fuzzy/ML entity resolution (deterministic exact match only); a full UI (CLI only). **Known limits, not descoped**: GitHub live fetch is unauthenticated (60 req/hr/IP); LinkedIn blocks unauthenticated live fetch (degrades to empty) — its cached JSON export is the reliable path.